1. Purpose

This document defines the persistent data model for the AI Multi-Agent Candidate Evaluation System. It translates the PRD, Technical Design, and Agent Specification into entities, relationships, constraints, and lifecycle rules. The design is intentionally suitable for a PostgreSQL MVP.

2. Design Principles

Evidence is a first-class entity. Agent opinions are stored independently. Debate messages and opinion revisions are immutable historical records. Final decisions retain traceability to the evidence and reasoning that produced them. Candidate data is isolated by evaluation. The database must make it difficult to accidentally leak one agent's independent conclusion into another agent's context.

3. Core Entities

The MVP uses these primary entities: Candidate, Document, Job, JobRequirement, Evaluation, CandidateProfile, ProfileItem, Evidence, RequirementEvidenceMapping, AgentEvaluation, AgentFinding, Debate, DebateMessage, OpinionRevision, FinalDecision, DecisionFactor, and ExecutionEvent.

4. Candidate

Candidate represents the person being evaluated. Suggested fields: id UUID primary key; display_name; external_reference optional; created_at; updated_at. Do not store unnecessary personal information. Candidate records are separated from Evaluation so the same candidate can potentially be evaluated against different jobs.

5. Document

Document represents an uploaded source. Fields: id; candidate_id; evaluation_id; document_type; original_filename; mime_type; file_size; storage_reference; processing_status; extracted_text; created_at; processed_at. document_type values: RESUME, TRANSCRIPT, OTHER. A document belongs to exactly one evaluation in the MVP to simplify isolation.

6. Job

Job represents the role against which a candidate is evaluated. Fields: id; title; description; created_at; updated_at. A Job can have many JobRequirements and can be reused across evaluations.

7. JobRequirement

Fields: id; job_id; requirement_text; category; priority; created_at. category can be TECHNICAL, EXPERIENCE, BEHAVIORAL, EDUCATION, OTHER. priority: MUST_HAVE, PREFERRED, OPTIONAL. Requirement extraction should preserve the original job-description wording.

8. CandidateProfile

CandidateProfile is the neutral factual representation shared by all agents. Fields: id; evaluation_id unique; summary_facts; created_at; updated_at. It must not contain hiring recommendations or agent conclusions. It can reference ProfileItems and Evidence.

9. ProfileItem

Fields: id; profile_id; item_type; value; claim_status; evidence_ids or a normalized join table; created_at. item_type examples: SKILL, EXPERIENCE, PROJECT, EDUCATION, CERTIFICATION, CLAIM, TECHNOLOGY. claim_status:

FACT, CANDIDATE_CLAIM, INFERENCE, UNKNOWN. The Profile Builder must not turn a candidate claim into a verified fact.

10. Evidence

Evidence is the central traceability entity. Fields: id; evaluation_id; document_id; source_type; page; section; paragraph; timestamp; speaker; exact_quote; structured_fact; evidence_type; extraction_confidence; validation_status; created_at. evidence_type: DIRECT_QUOTE, STRUCTURED_FACT, INFERENCE, UNSUPPORTED. validation_status: VALID, INVALID, PENDING. Evidence must always point to its source document.

11. Evidence Validation

An Evidence record is not considered verified merely because an LLM produced it. The Evidence Service validates the quote or fact against the stored source. Invalid evidence must not be displayed as verified support. If a finding references invalid evidence, the finding should be marked unsupported or sent through a controlled retry.

12. RequirementEvidenceMapping

This join entity maps role requirements to candidate evidence. Fields: id; requirement_id; evidence_id; status; rationale; created_at. status: SUPPORTED, PARTIALLY_SUPPORTED, NOT_SUPPORTED, UNKNOWN. This mapping describes evidence coverage and is not itself the final hiring decision.

13. Evaluation

Evaluation is the top-level execution container. Fields: id; candidate_id; job_id; status; pipeline_version; started_at; completed_at; created_at. status: CREATED, PROCESSING, INDEPENDENT_COMPLETE, DEBATE_COMPLETE, DECISION_COMPLETE, COMPLETED, FAILED, PARTIAL. All evaluation-specific records must carry evaluation_id directly or be reachable through a parent with enforced foreign keys.

14. AgentEvaluation

Fields: id; evaluation_id; agent_type; assessment; recommendation; confidence; confidence_reason; strengths; concerns; uncertainties; created_at; prompt_version; model_name. agent_type: TECHNICAL, HR_CULTURE, HIRING_MANAGER, SKEPTIC. recommendation: STRONG_HIRE, HIRE, CONSIDER, WEAK_CONSIDER, REJECT. confidence: HIGH, MEDIUM, LOW. There must be at most one independent AgentEvaluation per agent_type per evaluation unless an explicit retry/version mechanism is used.

15. AgentFinding

Fields: id; agent_evaluation_id; title; statement; severity; reasoning; confidence; created_at. Findings reference evidence through AgentFindingEvidence. severity can be POSITIVE, NEUTRAL, LOW_CONCERN, MEDIUM_CONCERN, HIGH_CONCERN. A finding without valid evidence must explicitly state insufficient evidence.

16. AgentFindingEvidence

Many-to-many join between AgentFinding and Evidence. Fields: finding_id; evidence_id. The join enables a finding to be supported by multiple source items and an evidence item to support findings from different evaluators.

17. Debate

Debate represents the post-independence interaction phase. Fields: id; evaluation_id unique; status; current_round; max_rounds; started_at; completed_at. status: NOT_STARTED, ACTIVE, COMPLETED, FAILED. Debate creation is permitted only after the independent stage is complete or explicitly marked partial.

18. DebateMessage

Fields: id; debate_id; round_number; agent_type; message_type; statement; responding_to_message_id nullable; evidence_ids through join table; confidence; created_at. message_type: CHALLENGE, RESPONSE, REVISION, CLARIFICATION. responding_to_message_id is mandatory for RESPONSE and should normally be mandatory for REVISION when the revision was triggered by debate.

19. OpinionRevision

Fields: id; evaluation_id; agent_evaluation_id; original_position; revised_position; original_confidence; revised_confidence; reason; created_at. Link triggering debate messages through OpinionRevisionMessage. A revision must never silently overwrite the original AgentEvaluation. The original independent opinion remains preserved.

20. FinalDecision

Fields: id; evaluation_id unique; recommendation; confidence; confidence_reason; decision_rationale; alternative_recommendation; alternative_reason; created_at; model_name; prompt_version. The FinalDecision is generated by a separate decision-stage call and must not be stored as a computed average of evaluator scores.

21. DecisionFactor

Fields: id; final_decision_id; factor_type; statement; importance; reasoning; created_at. factor_type: REQUIREMENT_FIT, EVIDENCE_STRENGTH, CONCERN, CONTRADICTION, DEBATE_OUTCOME, CONFIDENCE, UNCERTAINTY, OTHER. Decision factors reference AgentFindings and/or Evidence through join tables.

22. UnresolvedDisagreement

Fields: id; final_decision_id; topic; agents_involved; summary; impact; evidence_ids; created_at. This entity makes unresolved disagreement explicit in the final report rather than hiding it in free-form prose.

23. ExecutionEvent

ExecutionEvent provides auditability. Fields: id; evaluation_id; stage; agent_type nullable; event_type; status; metadata; started_at; completed_at. Useful event types include DOCUMENT_PARSED, PROFILE_BUILT, AGENT_STARTED, AGENT_COMPLETED, EVIDENCE_VALIDATED, DEBATE_STARTED, DEBATE_MESSAGE_CREATED, REVISION_CREATED, DECISION_STARTED, DECISION_COMPLETED, ERROR.

24. Key Relationships

Candidate 1—M Evaluation. Job 1—M Evaluation. Evaluation 1—M Document. Evaluation 1—1 CandidateProfile. CandidateProfile 1—M ProfileItem. Document 1—M Evidence. Job 1—M JobRequirement. JobRequirement M—M Evidence through RequirementEvidenceMapping. Evaluation 1—M AgentEvaluation. AgentEvaluation 1—M AgentFinding. AgentFinding M—M Evidence. Evaluation 1—1 Debate. Debate 1—M DebateMessage. AgentEvaluation 1—M OpinionRevision. Evaluation 1—1 FinalDecision. FinalDecision 1—M DecisionFactor.

25. Isolation Constraints

Every evaluation-specific row must be scoped to one evaluation. Queries that load Evidence for an AgentEvaluation must join through the same evaluation_id. The application should reject any attempt to pass an Evidence ID belonging to another evaluation. Agent outputs must never be queried into the independent context except as the output of the current agent's own call. Database constraints and service-layer checks should both enforce this.

26. Immutability Rules

Independent AgentEvaluation records should be treated as immutable after creation. Corrections or retries create a new execution/version record rather than silently rewriting historical reasoning. DebateMessage records are append-only. OpinionRevision records are append-only. The final decision may be regenerated as a new decision version if the product supports re-running; the prior decision should remain auditable.

27. Suggested Indexes

Index evaluation_id on all evaluation-scoped tables. Index document_id on Evidence. Index job_id on JobRequirement. Index agent_type + evaluation_id on AgentEvaluation. Index debate_id + round_number on DebateMessage. Index final_decision_id on DecisionFactor. Index evidence validation_status for validation workflows. Add foreign-key indexes where PostgreSQL does not create them automatically.

28. Lifecycle

Created Evaluation → documents uploaded → documents parsed → evidence extracted → profile built → requirements extracted → independent agents executed → evidence validated → independent stage complete → debate created → debate rounds → revisions recorded → final decision → report. A failure at any stage should preserve the records already created and mark the stage/evaluation status accordingly.

29. Data Integrity Rules

Required fields must be validated before persistence. Enum fields should use PostgreSQL enums or constrained text values. Foreign keys must use restrictive behavior where deletion could break traceability. Prefer soft deletion or archival for documents and evaluations rather than destructive deletion when auditability is required.

30. Privacy and Retention

Store only information needed for evaluation. Keep API keys out of the database. Avoid unnecessary raw candidate data in logs. If the system is deployed beyond a classroom demo, define retention and deletion policies before storing real candidate documents.

31. Example Traceability Query

The system should be able to answer: “Why did the final decision contain this concern?” by traversing FinalDecision → DecisionFactor → AgentFinding → AgentFindingEvidence → Evidence → Document → exact_quote. The same graph should support UI interactions such as clicking a decision factor to open its source.

32. MVP Database Recommendation

Use PostgreSQL with UUID primary keys, JSONB only for genuinely flexible fields, and normalized tables for relationships that must be queried. Do not put the entire candidate profile, all agent outputs, and the entire debate into one JSON blob. Structured entities are necessary for filtering, traceability, and testing.

33. Completion Criteria

The data model is ready for implementation when every PRD-required object has a persistent representation; agent independence can be enforced through evaluation-scoped records; evidence can be validated and traced to source; debate relationships can be represented; revisions preserve history; final decisions can reference factors and evidence; and the report can be generated entirely from stored structured data.